Simplified Data Partitioning in a Consistent Hashing Based Sharding Implementation

Narayanan Venkateswaran, Dr Suvamoy Changder

***Abstract*—Sharding implementations use consistent hashing for distributing a database uniformly across the servers in the topology. Each data item in the database is identified uniquely by a sharding key. The sharding keys are hashed into a hash ring. This hash ring is split into equal sized partitions and each server is allocated an equal number of partitions. Each partition represents a portion of the database, referred to as a virtual shard. Maintaining virtual shards and presenting an unified picture of the database to the user requires complicated logic. This paper provides a mathematical analysis of the existing method of sharding and provides a more simple, efficient and flexible alternative that does not require virtual shards. The proposed method is compared to the existing method by simulating the algorithms on multiple datasets and it is shown that the expected distribution is obtained.**

I. Introduction

Sharding, or horizontal scaling, partitions the data set and distributes the partitions over multiple commodity servers [1][2]. Sharding is used in several datastores like Dynamo [3], Riak [4], Cassandra [5], MongoDB [6][7], for horizontally scaling data across commodity servers. The Physical representation of a shard is specific to both the datastore architecture and the particular sharding implementation.

In order to create a uniform distribution and for ease of load balancing, sharding implementations store multiple partitions of a data set on a server. Each of these partitions are referred to as virtual shards. Figure 1 shows an example of three virtual shards of a relational database EMP.

The user views the same relational structure in any shard that is accessed. This is because the sharding implementation is transparent to a user of the topology, and ensures that there is no difference in a query fired in a sharded or an unsharded environment. This implies that a projection fetching data about an employee from the EMP database, $\prod_{empid:empid=ID}(EMP)$, should work on any shard it is fired on.

However, most relational stores do not support multiple databases on the same database server as represented by the implementation in figure 1. Other workarounds for supporting this transparency like, query rewrite and query interception are not features commonly exposed by most relational stores. The lack of support for these features from the relational stores means that complex application logic is required to workaround their absence. Implementing these features in the application, increases the complexity of the setup making it more prone to errors. It also increases the cost of creating and maintaining virtual shards. To overcome these difficulties this paper proposes an efficient alternative that creates accurate partitions of the data in the servers without the use of virtual shards.

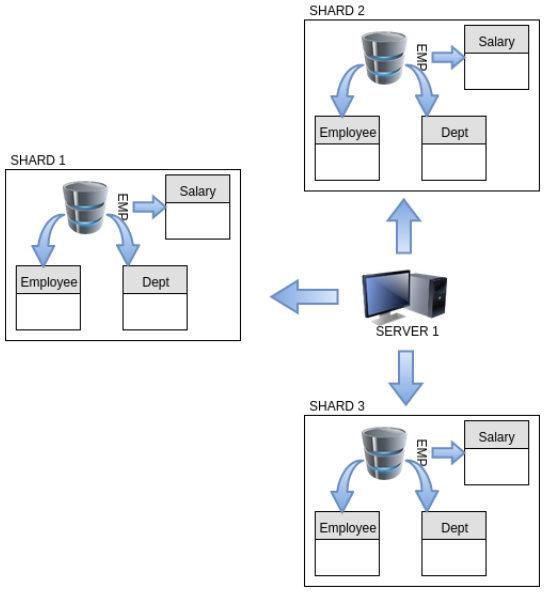

Fig. 1: Virtual Shards in a relational store.

In the rest of this paper, Section 2 discusses about the virtual shard implementation used in popular datastores and the related problems with supporting virtual shards. Section 3 talks about the proposed alternative to the use of virtual shards. Section 4 simulates the existing solution against the proposed method for a variety of datasets.

II. Existing Solution

This section looks at the basic principle underlying virtual shards and the related algorithms in detail.

A. Creating virtual shards

Existing sharding implementations use consistent hashing for distributing the data among the shards [8]. Consistent hashing ensures that the load is distributed uniformly and also ensures that the addition of a new server does not require rehashing all the data items in the topology [9].

A data item is placed in a server based on the value of an attribute of the data item. This attribute is called the sharding key. A sharding function is used to translate the value of a sharding key into the location of a server in the topology.

The key space of a sharding function refers to the list of possible values the sharding function can take. The key space

978-1-5090-1134-6/17/$31.00 ©2017 IEEE

Algorithm 1 creating shards

```
1: function CREATESHARD(S, V)        ▷ S[1..n] - Servers in the partitioning topology, V - Number of Virtual Shards
2:     virtual_partition_size ← key_space/V        ▷ Calculate size of each virtual partition
3:     cnt ← 0
4:     for i = 1 to n do
5:         for j = 1 to v do
6:             LB[i][j] ← cnt * virtual_partition_size
7:             cnt ← cnt + 1
8:         end for
9:     end for
10: return LB        ▷ LB[1..n][1..v] - Lower Bound for each server
11: end function
```

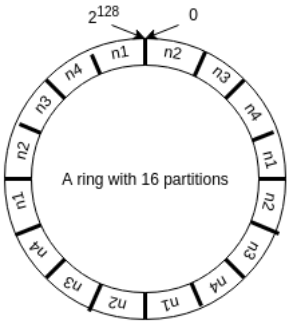

Fig. 2: Splitting a hash ring.

is divided into multiple partitions and distributed among the servers of the topology. This is shown in figure 2. In the figure, the hash ring represents the key space. The hash ring is divided into multiple partitions and each partition is distributed to one of the four server nodes - n1, n2, n3 and n4. Cryptographic hashing functions like MD5 [10], SHA-1 [11] are natural candidates as sharding functions, because of their randomness and spread over their key space.

The logic for partitioning the key space of a sharding function is shown in algorithm 1. The algorithm partitions the key space of a sharding function into multiple equal sized partitions. Each server in the topology gets an equal share of the partitions.

The partitioned key spaces in each of the servers represents the range of keys handled by each virtual shard. Each server is allocated multiple partitions of the key space. Each of these representing a virtual shard in the server.

1) Achieving a uniform distribution: From section II-A it can be seen that the virtual shards correspond to equal sized portions of the consistent hashing ring. Since each server is allocated multiple virtual shards, the larger the number of virtual shards (NV) in a server(S), the larger is the probability of a key (K) being in a server (S). This is shown in equation 1.

$$\boxed{P(K \in S) \propto NV(S)} \tag{1}$$

As the probability of keys being in a server increases, the amount of data allocated to a server also increases. Thus the size of data in a server is directly proportional to the probability of keys being allocated to a server. If the size of the data in a server S is given by D(S), then the above is described by equation 2,

$$\boxed{D(S) \propto P(K \in S)} \tag{2}$$

From equation 1 the probability of a key K being in two servers S_1 and S_2 is given by, equation 3 and equation 4 respectively.

$$\boxed{P(K \in S_1) \propto NV(S_1)} \tag{3}$$

$$\boxed{P(K \in S_2) \propto NV(S_2)} \tag{4}$$

From 2 the size of the data present in two servers S_1 and S_2 is given by, equation 5 and equation 6.

$$\boxed{D(S_1) \propto P(K \in S_1)} \tag{5}$$

$$\boxed{D(S_2) \propto P(K \in S_2)} \tag{6}$$

In an uniform distribution each server in the topology is allocated an equal number of data items. When the number of data items in each server are equal, each server should have an equal probability of being chosen for a data item's key value as shown in equation 7.

$$\boxed{P(K \in S_1) = P(K \in S_2) = P(K \in S_3)...} \tag{7}$$

From equation 3 and equation 3 we can derive equation 8.

$$\boxed{\frac{P(K \varepsilon S_1)}{P(K \varepsilon S_2)} = \frac{NV(S_1)}{NV(S_2)}} \tag{8}$$

From equation 7 and equation 8 we can derive equation 9.

$$\boxed{\frac{P(K\varepsilon S_1)}{P(K\varepsilon S_2)} = \frac{DS(S_1)}{DS(S_2)}} \quad (9)$$

From equation 8 and equation 9 we can derive equation 10.

$$\boxed{\frac{NV(S_1)}{NV(S_2)} = \frac{DS(S_1)}{DS(S_2)}} \quad (10)$$

From equation 9 and equation 10 we can derive equation 11 when the number of virtual shards allocated to each server are equal.

$$\boxed{DS(S_1) = DS(S_2)} \quad (11)$$

The existing solution leverages equation 10 and creates a uniform distribution by distributing equal number of virtual shards to each server.

B. Shortcomings of using virtual shards

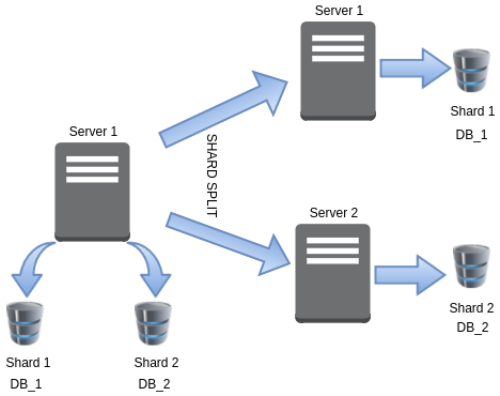

Fig. 3: Multiple shards in a server.

The sharding implementation needs to be transparent to the end user. The user fires a query unaware of whether the underlying database object is sharded and sees the same result in all cases. The setup created for storing data should be insulated from the end user who sees an unified picture of the data that exists [12].

Maintaining virtual shards in a server is efficient and practical when, the data store offers support for creating and maintaining such a transparent implementation. In a relational store, for example, all the data in a shard can be consolidated in a database. However, most relational stores allow only one instance of the database in a given server [13]. In order to implement virtual shards in this case, the sharding implementation will need to rely on workarounds like storing the shards in different database names on a given server. In order to support virtual shards, each shard is stored in a different database on the same server.

The figure 3 shows a setup where a database EMP is sharded. In this figure the shards are stored in databases that are named with the shard ID as the suffix (i.e.) shard 1 in DB_1, shard 2 in DB_2 and so on. Notice that the implementation needs to create and track database names, everytime a shard is created. Although the setup shown in figure 3 allows for ease of splitting the shard between servers it results in query rewrites.

For example in the figure 3, a query for a table in database DB will have to be rewritten for operating on the databases DB_1 and DB_2. This is seen in the projection in equation 12.

$$\boxed{\prod_{A}(DB.table) \rightarrow \prod_{A_1}(DB_1.table) + \prod_{A_2}(DB_2.table)} \quad (12)$$

If the datastore offers a way of consolidating all the data in a shard together, and allows maintaining multiple such consolidations for the same shard in a single server, this can be directly used for implementing virtual shards. Otherwise storing multiple shards on the server requires working around the limitation of one database per server. Thus the efficiency of a virtual shard implementation depends on the support available from the underlying datastore.

III. Proposed Solution

This section discusses how to create a uniform distribution of a database across a set of servers without the use of virtual shards. In addition to creating a uniform distribution the proposed method is also capable of creating customizable load distributions, in which, each server gets a user specified proportion of the database. Finally it is also shown that the same principles can be applied while resharding a database in a server.

A. Basic Principle

In this section we establish the relationship between the ratio of the hash ring partitions associated with the servers and the size of the data in the servers.

1) Virtual shards and key space size: The key space size refers to the number of keys from the hash ring allocated to each server.

The number of virtual shards in each server determines the size of the key space (KS) allocated to each server. The larger the number of virtual shards allocated to each server, the larger is the key space size in the server. This leads to equations 13 and 14.

$$\boxed{KS(S_1) \propto NV(S_1)} \quad (13)$$

$$\boxed{KS(S_2) \propto NV(S_2)} \quad (14)$$

From equations 13 and 14, it can be seen that the ratio of the keyspace sizes allocated to each server is equal to the ratio of the number of virtual shards allocated to each of them. This is shown in equation 15.

$$\boxed{\frac{KS(S_1)}{KS(S_2)} = \frac{NV(S_1)}{NV(S_2)}} \quad (15)$$

Using equations 10 and equation 15 we can infer equation 16.

$$\boxed{\frac{KS(S_1)}{KS(S_2)} = \frac{DS(S_1)}{DS(S_2)}} \quad (16)$$

From equation 16 it can be seen that when the key space sizes allocated to each server are equal, then the datasize allocated to them are also equal.

2) Creating customized shards: From equation 16 allocating appropriate key space sizes helps vary the size of data allocated to the shards.

If the threshold of the data size, for efficient operation, in a server is known it should be possible to create a more intelligent distribution of the key space among the servers. The threshold of a server points to the size of the data in the server beyond which its performance starts to dip.

The data size thresholds of a server can be found by benchmarking the hardware configuration and can be fine tuned over the operation of the topology. Discussing the factors that influence the threshold of a system is beyond the scope of this paper. When the thresholds of all the servers in a topology are equal, an uniform distribution is most efficient.

Let the threshold of the data sizes in the servers be given by T_1, T_2, T_3,...T_n. The corresponding ratio of the key space sizes can be calculated using equation 17.

$$\boxed{R_k = \frac{T_k}{\sum_{i=1}^{n} T_i}(100)} \quad (17)$$

If the size of the hash space is given by H, the key space (KS_k) for server S_k is given by equation 18.

$$\boxed{KS_k = \frac{R_k}{\sum_{i=1}^{n} R_i}(H)} \quad (18)$$

The equation 18 can be used to create a key space associated with a server which helps in creating a data distribution proportionate to the threshold capacity of the server.

B. Creating Shards

The algorithm 2 shows how the key space can be partitioned without the need to create virtual shards. The algorithm creates a distribution proportional to the data size of optimum performance of each server. when the thresholds of the servers are equal, the key space sizes allocated to each server are equal. From the equation 16 it can be seen that, equal thresholds result in an uniform distribution.

In the algorithm 2 the number of key space partitions is equal to the number of servers in the topology. Each server gets only one key space partition and as a direct consequence only one shard. The key space depends on the sharding algorithm being used. When using the MD5 hash, the key space size used is 2^{128}.

C. Resharding

Resharding in the presence of virtual shards involves directly moving them between servers. In the proposed method each server has only one shard.

Shard splitting moves a portion of the data from the source to the destination server. In order to move a portion of the data, the key space associated with this shard needs to be split. For example, assuming that a new shard is created, to split 30% of the data from the original shard whose range of values in the key space is given by [LB_1, LB_2), into a new shard, the new upper bound is calculated using the formula given in equation 19.

$$\boxed{LB_{split} = LB_1 + (70/100)(LB_2 - LB_1)} \quad (19)$$

The new key space ranges after applying equation 19 are given by [LB_1, LB_{split}) and [LB_{split}, LB_2).

Once the new key space ranges have been determined, the data in the original shard will need to be split accordingly. For example, assuming that the datastore in this case is relational and the shard were maintained in a database, a projection followed by a deletion operation can be employed to move the appropriate data into the new server.

The data that will be retained in this server is given by $\prod_{K:K >= LB_1 \; and K < LB_{split}}(R)$

The data that will be moved into a new server is given by $\prod_{K:K >= LB_{split} \; and K < LB_2}(R)$

Once the copies have been consistently created, the extra data can be deleted and the data can be synced with the original server.

In the following section we shall compare distributions created with and without the use of virtual shards to quantitatively verify the performance of the proposed method.

IV. Quantitative Simulation

In this section the distribution of data across the servers with the existing method (using virtual shards) is compared with the distribution of data across the servers with the proposed method (without virtual shards), to verify that both of them result in an uniform distribution. A shard split operation is attempted on shards created from both the methods and it is shown that the shards are split in the desired proportion.

A. Setup

In order to test the performance of the algorithm, it is assumed that there are a set of 8 servers with identical data size thresholds. One additional server is added to the topology while performing a split.

The output of the MD5 [10] algorithm is a 128-bit value. The range of possible values of the MD5 algorithm, represented as a ring, is given by 0 - 2^{128}-1. The hash ring is

Algorithm 2 creating shards

```
1:  function CreateShards(S, T, KS)          ▷ S-Servers, T-Thresholds, KS-key space size
2:      T_sum ← Σ_{i=1}^{n} T_i              ▷ Sum of Thresholds
3:      for i = 1 to n do
4:          key_space_ratio ← (T_i / T_sum) × 100      ▷ Percentage of key space allocated
5:          shard_size ← (key_space_ratio / 100) × KS  ▷ key space size allocated
6:          if i = 1 then                    ▷ First shard
7:              LB[i] ← shard_size
8:          else
9:              LB[i] = LB[i − 1] + shard_size
10:         end if
11:     end for
12: return LB
13: end function
```

TABLE I: Distribution of 10000000 values using virtual shards

Server	Exp	Sample 1	Sample 2	Sample 3
S1	1250000	1328060	1327555	1327796
S2	1250000	1251714	1252426	1251416
S3	1250000	1248681	1251312	1250727
S4	1250000	1248583	1248709	1249632
S5	1250000	1249275	1248562	1250338
S6	1250000	1248658	1250018	1249554
S7	1250000	1251103	1249969	1250410
S8	1250000	1173926	1171449	1170127

divided into 2^7 virtual shards. When there are 8 servers, each server gets 2^4 virtual shards.

The data distribution of both the algorithms, are tested against three different datasets, each of size 10000000, to ensure uniform distribution in each case. The heterogeneity of the datasets, is for ensuring consistent performance in all use cases.

B. Comparing data distribution

The table I shows the distribution of 10 million values, when using virtual shards, across eight servers.

The figure 4 shows the data in table I plotted as a line graph.

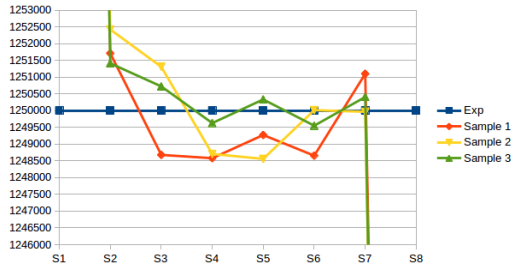

Fig. 4: Distribution using virtual shards

The table II shows the distribution of 10 million values, without using virtual shards, across eight servers.

TABLE II: Distribution of 10000000 values without virtual shards

Server	Exp	Sample 1	Sample 2	Sample 3
S1	1250000	1249899	1249115	1250298
S2	1250000	1251324	1252502	1250205
S3	1250000	1249125	1251096	1249626
S4	1250000	1248787	1249224	1250422
S5	1250000	1249376	1248944	1250566
S6	1250000	1248268	1250003	1250330
S7	1250000	1250893	1249261	1248668
S8	1250000	1252328	1249855	1249885

The figure 5 shows the data in table II plotted as a line graph.

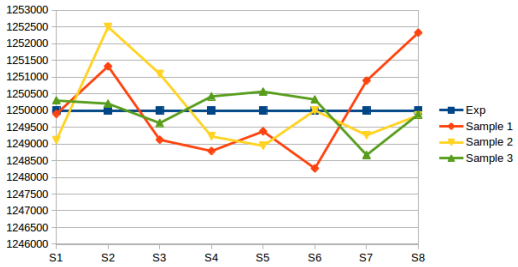

Fig. 5: Distribution without using virtual shards

From figure 5 it can be seen that the proposed solution creates a distribution of data items across the servers that matches the expected distribution better, than the solution that uses virtual shards (plotted in the figure 4).

C. Shard Split

A portion (30%) of the data in server 5 is split into a new server that is added to the topology.

The table III shows the results of splitting the data in server 5, as given in Sample 2 of table I.

The split is accomplished by moving 30% (approx 5) of the virtual shards in server 5 into another server. Note that moving

TABLE III: Splitting data using virtual shards

Server	Sample 2
S5	858402
S9	390160

TABLE IV: Splitting data without virtual shards

Server	Sample 2
S5	873361
S9	375863

the virtual shards this way results in moving 31.24% of the data values.

The table IV shows the results of splitting the data in server 5, as given in Sample 2 of table II.

The split is accomplished by splitting 30% of the key space, allocating it to the new server and projecting the tuples that fall in this 30% key space into the new server. Note that this results in moving 30.09% of the data. Thus this method results in a more accurate split.

V. Conclusion and Future Work

The algorithms that are used to shard the data, work by static partitioning of the data in a server into a number of virtual shards. Virtual shards offer advantages when the underlying datastore offers primitives that support efficient sharding operations. In the absence of these primitives, virtual shards become counter-productive. The paper proposes an alternative to creating and maintaining shards, without the use of virtual shards. The paper also verifies the efficieny of the proposed method, for both creating and maintaining shards, and finds that the proposed method performs satisfactorily and in some cases better than the existing methods that use virtual shards.

The algorithms presented in this paper talk about incorporating thresholds for creating flexible distributions of the database across the servers in the topology. Thresholds offer huge scope for creating accurate distributions in environments with elastic scalability like the cloud. Work needs to be done in the area of estimating thresholds automatically from the available processing power and using the thresholds to create flexible distributions across the servers in the topology.

References

[1] P. Kookarinrat and Y. Temtanapat. "Analysis of Range-Based Key Properties for Sharded Cluster of MongoDB". In: *Information Science and Security (ICISS), 2015 2nd International Conference on*. Dec. 2015, pp. 1–4. DOI: 10.1109/ICISSEC.2015.7370983.

[2] Man Qi et al. "Big Data Management in Digital Forensics". In: *Computational Science and Engineering (CSE), 2014 IEEE 17th International Conference on*. Dec. 2014, pp. 238–243. DOI: 10.1109/CSE.2014.74.

[3] Deniz Hastorun et al. "Dynamo: amazons highly available key-value store". In: *In Proc. SOSP*. 2007, pp. 205–220.

[4] *Riak Architecture*. http://docs.basho.com/riak/latest/ops/mdc/v3/architecture/. Accessed: 2016-02-16.

[5] *Cassandra Architecture*. https://docs.datastax.com/en/cassandra/2.0/cassandra/architecture/architectureIntro_c.html. Accessed: 2016-02-16.

[6] Chao-Wen Huang et al. "The improvement of auto-scaling mechanism for distributed database - A case study for MongoDB". In: *Network Operations and Management Symposium (APNOMS), 2013 15th Asia-Pacific*. Sept. 2013, pp. 1–3.

[7] Xiaolin Wang, Haopeng Chen, and Zhenhua Wang. "Research on Improvement of Dynamic Load Balancing in MongoDB". In: *Dependable, Autonomic and Secure Computing (DASC), 2013 IEEE 11th International Conference on*. Dec. 2013, pp. 124–130. DOI: 10.1109/DASC.2013.49.

[8] David Karger et al. "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web". In: *In ACM Symposium on Theory of Computing*. 1997, pp. 654–663.

[9] G. Swart. "Spreading the load using consistent hashing: a preliminary report". In: *Parallel and Distributed Computing, 2004. Third International Symposium on/Algorithms, Models and Tools for Parallel Computing on Heterogeneous Networks, 2004. Third International Workshop on*. July 2004, pp. 169–176. DOI: 10.1109/ISPDC.2004.47.

[10] R. Rivest. *The MD5 Message-Digest Algorithm*. RFC 1321. 1992.

[11] D. Eastlake. *US Secure Hash Algorithm 1 (SHA1)*. RFC 3174. 2001.

[12] Lars Thalmann Charles Bell Mats Kindahl. *MySQL High Availability, 2nd Edition*. O'Reilly Media, 2014.

[13] *MySQL Documentation*. http://dev.mysql.com/doc/refman/5.7/en/. Accessed: 2016-04-07.

[14] *MongoDB Sharding Introduction*. https://docs.mongodb.org/manual/core/sharding-introduction/. Accessed: 2016-02-16.

[15] D. Agarwal and S.K. Prasad. "AzureBench: Benchmarking the Storage Services of the Azure Cloud Platform". In: *Parallel and Distributed Processing Symposium Workshops PhD Forum (IPDPSW), 2012 IEEE 26th International*. May 2012, pp. 1048–1057. DOI: 10.1109/IPDPSW.2012.128.

[16] S. Ramanathan, S. Goel, and S. Alagumalai. "Comparison of Cloud database: Amazon's SimpleDB and Google's Bigtable". In: *Recent Trends in Information Systems (ReTIS), 2011 International Conference on*. Dec. 2011, pp. 165–168. DOI: 10.1109/ReTIS.2011.6146861.

[17] Kristina Chodorow. *Scaling MongoDB*. O'Reilly Media, 2011.